

PostgreSQL Interview Questions

Concepts, Queries & Administration

77 questions with answers

Interview Preparation Guide

PostgreSQL Concepts

Q1. What is PostgreSQL?

PostgreSQL is a powerful open-source object-relational database management system (ORDBMS) known for standards compliance, extensibility, ACID transactions, and advanced features like JSONB, window functions, CTEs, and full-text search.

Q2. What are ACID properties?

Atomicity (all-or-nothing transactions), Consistency (valid state transitions), Isolation (concurrent transactions don't interfere), Durability (committed data survives crashes).

Q3. What is the difference between PostgreSQL and MySQL?

PostgreSQL is object-relational, more standards-compliant, supports advanced types (JSONB, arrays, custom types), richer concurrency (MVCC), and more complex queries. MySQL is traditionally simpler and was historically faster for read-heavy simple workloads.

Q4. What is a primary key?

A column (or set of columns) that uniquely identifies each row. It is NOT NULL and UNIQUE; a table can have only one primary key.

Q5. What is a foreign key?

A column that references the primary key of another table, enforcing referential integrity between related tables.

Q6. What is the difference between DELETE, TRUNCATE, and DROP?

DELETE removes rows (can use WHERE, logged, can be rolled back). TRUNCATE quickly removes all rows (faster, resets identity, minimal logging). DROP removes the entire table definition.

Q7. What is an index and why use it?

An index is a data structure (B-tree by default) that speeds up lookups and ordering at the cost of extra storage and slower writes.

Q8. What index types does PostgreSQL support?

B-tree (default), Hash, GIN (full-text/JSONB/arrays), GiST (geometric/range), BRIN (large ordered tables), and SP-GiST.

Q9. What is MVCC?

Multi-Version Concurrency Control: PostgreSQL keeps multiple row versions so readers don't block writers and vice versa, providing snapshot isolation.

Q10. What is VACUUM and why is it needed?

VACUUM reclaims storage from dead tuples left by MVCC updates/deletes, prevents transaction-ID wraparound, and updates statistics (with ANALYZE). Autovacuum automates this.

Q11. What is the difference between a clustered and non-clustered index?

PostgreSQL has no permanent clustered index; CLUSTER physically reorders a table by an index once. Indexes are otherwise secondary (non-clustered) structures pointing to heap tuples.

Q12. What are the different types of JOINS?

INNER JOIN (matching rows), LEFT/RIGHT OUTER JOIN (all rows from one side + matches), FULL OUTER JOIN (all rows both sides), CROSS JOIN (cartesian product), and SELF JOIN.

Q13. What is a view and a materialized view?

A view is a stored query that runs each time it is referenced. A materialized view stores the result physically and must be refreshed with REFRESH MATERIALIZED VIEW.

Q14. What is a CTE (Common Table Expression)?

A named temporary result set defined with WITH that improves readability and enables recursion (WITH RECURSIVE).

Q15. What is normalization?

Organizing tables to reduce redundancy and improve integrity, through normal forms (1NF: atomic values; 2NF: no partial dependency; 3NF: no transitive dependency; BCNF).

Q16. What is the difference between WHERE and HAVING?

WHERE filters rows before grouping; HAVING filters groups after GROUP BY (it can use aggregates).

Q17. What is a transaction and how do you control it?

A unit of work executed atomically. Controlled with BEGIN, COMMIT, ROLLBACK, and SAVEPOINT for partial rollbacks.

Q18. What are isolation levels in PostgreSQL?

READ COMMITTED (default), REPEATABLE READ, and SERIALIZABLE. (READ UNCOMMITTED behaves like READ COMMITTED.) They control visibility of concurrent changes.

Q19. What is the difference between UNION and UNION ALL?

UNION combines result sets and removes duplicates (sorts); UNION ALL keeps all rows including duplicates and is faster.

Q20. What is a sequence and SERIAL/IDENTITY?

A sequence generates unique numbers. SERIAL is shorthand creating an integer column backed by a sequence; GENERATED AS IDENTITY is the SQL-standard, preferred modern approach.

Q21. What is EXPLAIN / EXPLAIN ANALYZE?

EXPLAIN shows the query planner's chosen execution plan and cost estimates; EXPLAIN ANALYZE actually runs the query and reports real timing and row counts.

Q22. What is a trigger?

A function automatically executed in response to INSERT/UPDATE/DELETE events on a table, defined with CREATE TRIGGER calling a trigger function.

Q23. What is the difference between CHAR, VARCHAR, and TEXT?

CHAR(n) is fixed-length (blank-padded); VARCHAR(n) is variable with a length limit; TEXT is unlimited variable length. In PostgreSQL all perform similarly; TEXT/VARCHAR are usually preferred.

Q24. What is JSONB and how does it differ from JSON?

JSON stores raw text (preserves formatting, slower to query). JSONB stores a decomposed binary form that is faster to query and supports indexing (GIN), at slightly higher write cost.

Basic Queries

Q25. Select all employees from the employees table.

```
SELECT * FROM employees;
```

Q26. Select only name and salary columns.

```
SELECT name, salary FROM employees;
```

Q27. Find employees with salary greater than 50000.

```
SELECT * FROM employees WHERE salary > 50000;
```

Q28. Find employees in the 'Sales' or 'Marketing' department.

```
SELECT * FROM employees WHERE department IN ('Sales', 'Marketing');
```

Q29. Find employees whose name starts with 'A'.

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

Q30. Find employees with no manager (NULL manager_id).

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

Q31. Sort employees by salary descending.

```
SELECT * FROM employees ORDER BY salary DESC;
```

Q32. Return the 5 highest-paid employees.

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

Q33. Return rows 11-20 (pagination).

```
SELECT * FROM employees ORDER BY id LIMIT 10 OFFSET 10;
```

Q34. Find employees hired between two dates.

```
SELECT * FROM employees  
WHERE hire_date BETWEEN '2023-01-01' AND '2023-12-31';
```

Q35. Select distinct department names.

```
SELECT DISTINCT department FROM employees;
```

Q36. Find employees with salary NOT between 30000 and 60000.

```
SELECT * FROM employees WHERE salary NOT BETWEEN 30000 AND 60000;
```

Aggregations & Grouping

Q37. Count the total number of employees.

```
SELECT COUNT(*) FROM employees;
```

Q38. Find the average salary per department.

```
SELECT department, AVG(salary) AS avg_salary  
FROM employees  
GROUP BY department;
```

Q39. Find departments with more than 10 employees.

```
SELECT department, COUNT(*) AS cnt  
FROM employees  
GROUP BY department  
HAVING COUNT(*) > 10;
```

Q40. Find the highest and lowest salary in the company.

```
SELECT MAX(salary) AS highest, MIN(salary) AS lowest FROM employees;
```

Q41. Find total salary expense per department, highest first.

```
SELECT department, SUM(salary) AS total  
FROM employees  
GROUP BY department  
ORDER BY total DESC;
```

Q42. Count employees per department, only departments starting with 'S'.

```
SELECT department, COUNT(*)  
FROM employees  
WHERE department LIKE 'S%'  
GROUP BY department;
```

Q43. Find the number of distinct departments.

```
SELECT COUNT(DISTINCT department) FROM employees;
```

Q44. Round the average salary to 2 decimals per department.

```
SELECT department, ROUND(AVG(salary), 2) AS avg_salary  
FROM employees  
GROUP BY department;
```

Joins & Subqueries

Q45. Join employees with their department names.

```
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

Q46. List all departments, including those with no employees.

```
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.id;
```

Q47. Find employees earning more than the company average.

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q48. Find employees who earn more than their department's average.

```
SELECT e.* FROM employees e
WHERE salary > (
    SELECT AVG(salary) FROM employees
    WHERE dept_id = e.dept_id
);
```

Q49. Find departments that have no employees.

```
SELECT d.* FROM departments d
LEFT JOIN employees e ON e.dept_id = d.id
WHERE e.id IS NULL;
```

Q50. Self join: list employees with their manager's name.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

Q51. Find employees who work on at least one project (EXISTS).

```
SELECT * FROM employees e
WHERE EXISTS (
    SELECT 1 FROM projects p WHERE p.emp_id = e.id
);
```

Q52. Find customers who never placed an order.

```
SELECT c.* FROM customers c
WHERE c.id NOT IN (SELECT customer_id FROM orders);
```

Q53. Total order amount per customer (join + group).

```
SELECT c.name, SUM(o.amount) AS total  
FROM customers c  
JOIN orders o ON o.customer_id = c.id  
GROUP BY c.name;
```

Advanced Queries

Q54. Find the second highest salary.

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Or with a window function:
SELECT DISTINCT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 2;
```

Q55. Find the Nth highest salary using OFFSET.

```
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET (N - 1);
```

Q56. Rank employees by salary within each department.

```
SELECT name, department, salary,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk
FROM employees;
```

Q57. Find the top earner in each department.

```
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY department ORDER BY salary DESC) AS rn
    FROM employees
) t WHERE rn = 1;
```

Q58. Calculate a running total of salaries ordered by id.

```
SELECT id, salary,
       SUM(salary) OVER (ORDER BY id) AS running_total
FROM employees;
```

Q59. Find duplicate emails in a users table.

```
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Q60. Delete duplicate rows keeping the lowest id.

```
DELETE FROM users a
USING users b
WHERE a.id > b.id AND a.email = b.email;
```

Q61. Use a CTE to find above-average earners.

```
WITH avg_cte AS (
    SELECT AVG(salary) AS avg_sal FROM employees
)
SELECT e.* FROM employees e, avg_cte
WHERE e.salary > avg_cte.avg_sal;
```

Q62. Recursive CTE: generate numbers 1 to 10.

```
WITH RECURSIVE nums AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 10
)
SELECT n FROM nums;
```

Q63. Pivot: count employees by department using FILTER.

```
SELECT
    COUNT(*) FILTER (WHERE department = 'Sales') AS sales,
    COUNT(*) FILTER (WHERE department = 'IT') AS it
FROM employees;
```

Q64. Use COALESCE to replace NULL commissions with 0.

```
SELECT name, COALESCE(commission, 0) AS commission
FROM employees;
```

Q65. Use CASE to bucket salaries into bands.

```
SELECT name,
    CASE
        WHEN salary < 30000 THEN 'Low'
        WHEN salary < 70000 THEN 'Mid'
        ELSE 'High'
    END AS band
FROM employees;
```

Q66. Find the difference between each employee's salary and the previous one.

```
SELECT name, salary,
    salary - LAG(salary) OVER (ORDER BY id) AS diff
FROM employees;
```

Q67. Get the most recent order per customer.

```
SELECT DISTINCT ON (customer_id) *
FROM orders
ORDER BY customer_id, order_date DESC;
```

DDL, DML & Administration

Q68. Create an employees table.

```
CREATE TABLE employees (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    department VARCHAR(50),  
    salary NUMERIC(10,2),  
    hire_date DATE DEFAULT CURRENT_DATE  
);
```

Q69. Add a new column to a table.

```
ALTER TABLE employees ADD COLUMN email VARCHAR(255);
```

Q70. Insert a new employee.

```
INSERT INTO employees (name, department, salary)  
VALUES ('Asha', 'IT', 65000);
```

Q71. Update an employee's salary by 10%.

```
UPDATE employees SET salary = salary * 1.10  
WHERE id = 5;
```

Q72. Upsert (insert or update on conflict).

```
INSERT INTO employees (id, name, salary)  
VALUES (5, 'Asha', 70000)  
ON CONFLICT (id)  
DO UPDATE SET salary = EXCLUDED.salary;
```

Q73. Create an index on the department column.

```
CREATE INDEX idx_emp_dept ON employees(department);
```

Q74. Add a foreign key constraint.

```
ALTER TABLE employees  
ADD CONSTRAINT fk_dept  
FOREIGN KEY (dept_id) REFERENCES departments(id);
```

Q75. Create a view of high earners.

```
CREATE VIEW high_earners AS  
SELECT name, salary FROM employees  
WHERE salary > 80000;
```

Q76. Grant SELECT on a table to a user.

```
GRANT SELECT ON employees TO analyst_user;
```

Q77. Write a simple trigger function and trigger.

```
CREATE OR REPLACE FUNCTION set_updated()  
RETURNS TRIGGER AS $$  
BEGIN  
    NEW.updated_at = NOW();  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;  
  
CREATE TRIGGER trg_updated  
BEFORE UPDATE ON employees  
FOR EACH ROW EXECUTE FUNCTION set_updated();
```